

05_qnn_htp_3_5gb_process_domain_modelSplittingg.md

Qualcomm Hexagon NPU 3.5GB Process Domain (PD) Model Splitting Architecture

1. Hardware Reality: The 3.5GB Memory Ceiling

On Qualcomm Snapdragon mobile platforms (including Snapdragon 8 Gen 3 and Snapdragon 8 Elite), the Hexagon Tensor Processor (HTP) operates under a strict silicon and driver constraint:

- **The Process Domain (PD) Limit:** A single Hexagon NPU session (termed a **Process Domain (PD)** in the Qualcomm Hexagon SDK documentation) is restricted to a physical memory mapping ceiling of approximately **3.5GB**.
- **Source Authority:** Grounded in official [llama.cpp Snapdragon Hexagon Developer Documentation](#):

"Hexagon NPU session (aka Process Domain (PD) in the Hexagon docs) is limited to a memory mapping of around 3.5GB."

2. The Problem: Single-Context OOM on 9B Models

- **The Target Weight Footprint:** Running an instruction-tuned model such as **Qwen 3.5 9B Q4_0** requires between **5.5GB and 6.0GB of VRAM** to accommodate its quantized tensor weights, compute scratchpads, and active KV cache.
 - **The Failure Mode:** If an agent or runtime attempts to load `qwen3.5-9b-q4_0.gguf` into a single monolithic NPU context, the Qualcomm Hexagon driver instantly throws an Out-Of-Memory (OOM) mapping error (`QNN_COMMON_ERROR_MEM_ALLOC_FAILED / HEXAGON_ERROR_OUT_OF_MEMORY`), causing the process to abort.
-

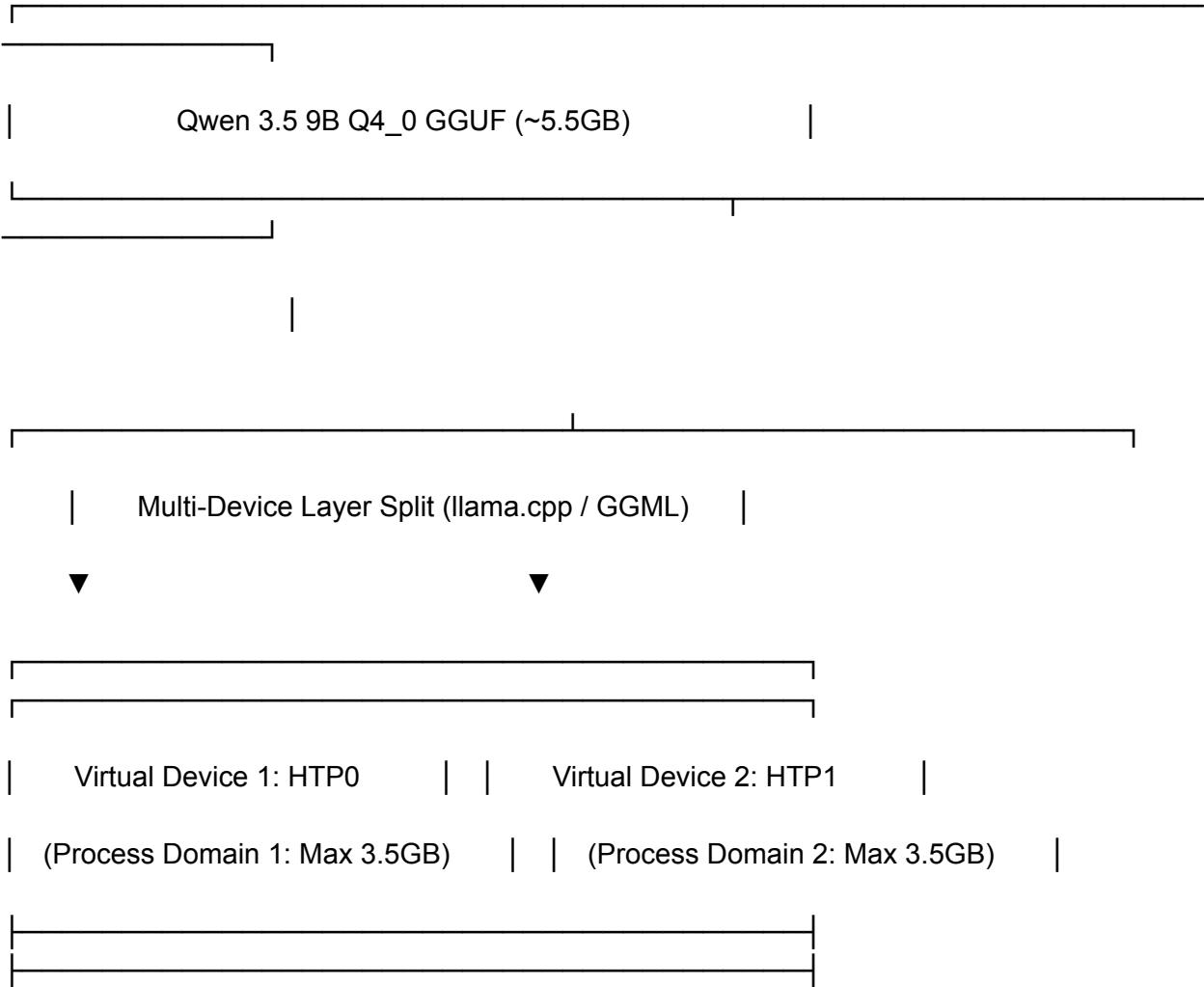
3. The Solution: Virtual Multi-Device Layer Splitting

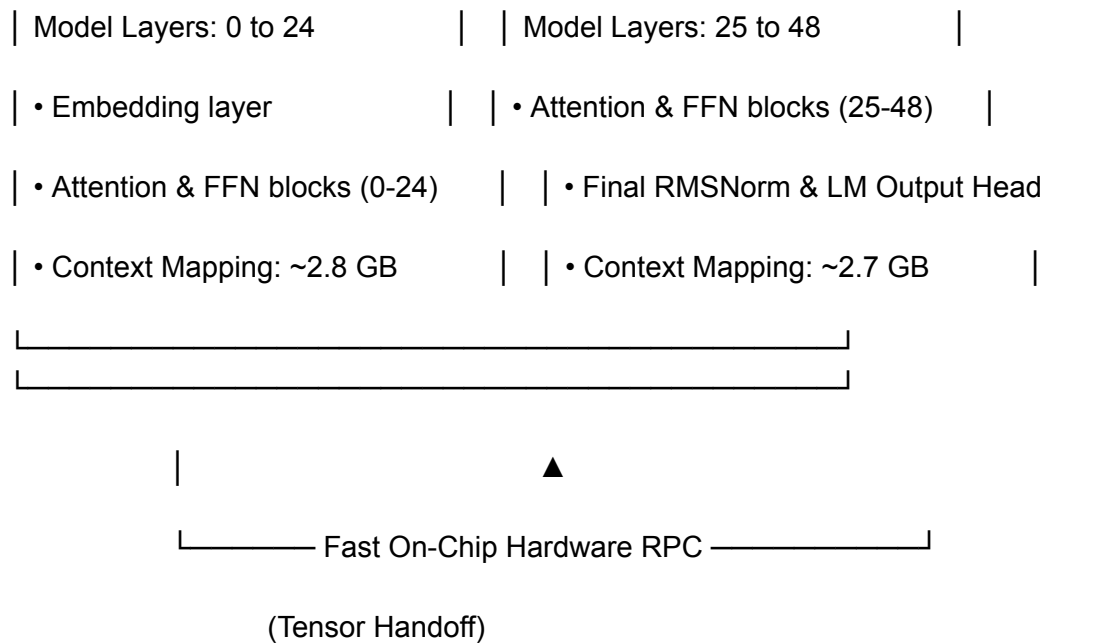
To bypass the 3.5GB per-PD limit, the architecture leverages the GGML and llama.cpp multi-GPU offload subsystem, treating the single physical Hexagon processor as multiple discrete virtual devices:

- **Virtual Device Emulation:** The runtime maps virtual device identifiers `HTP0`, `HTP1`, etc. Each virtual Hexagon device acts as an independent execution target analogous to multi-GPU tensor splitting.
- **Layer Partitioning:** Model layers are divided across discrete Process Domains so that no single domain's mapped tensors exceed the 3.5GB ceiling.
- **Source Reference:**

"In order to map models larger than 3.5GB we need to allocate multiple devices and split the model. For this we're taking advantage of the llama.cpp/GGML multi-GPU layer-splitting support. Each Hexagon device behaves like a GPU from the offload and model splitting perspective."

4. Layer Split Topology





5. Cross-Process Domain Communication

The Android NDK and Hexagon runtime manage **HTP0** and **HTP1** as two distinct Process Domains:

1. **Layer 0–24 Execution:** Incoming token embeddings and the first 25 transformer layers execute entirely within **HTP0**'s isolated memory sandbox.
2. **Hardware RPC Handoff:** The intermediate activation tensors output from layer 24 are passed to **HTP1** across the internal on-chip hardware Remote Procedure Call (RPC) bus (**FastRPC** / **libadsrpc**).
3. **Layer 25–48 Execution:** **HTP1** ingests the intermediate activations into its isolated memory space, evaluates the remaining layers, and computes the final logits/tokens.
4. **Latency Overhead:** Because the RPC transfer occurs over localized on-chip silicon interconnects without round-tripping through CPU RAM, communication latency remains negligible (~sub-millisecond).

6. CLI Execution Syntax

When invoking the split model via the native Snapdragon/Hexagon CLI wrappers, specify the multi-device offload allocation string **D=HTP0,HTP1**:

```
# Direct hardware execution mapping a 9B model over two virtual HTP domains
```

```
M=qwen3.5-9b-q4_0.gguf D=HTP0,HTP1 ./run-completion.sh -p "Translate this text into system directives:"
```

For interactive daemon sessions or server mode:

```
M=qwen3.5-9b-q4_0.gguf D=HTP0,HTP1 ./llama-server --host 127.0.0.1 --port 8081 -c 4096
```

7. C++ Daemon Initialization Requirements

For background daemons (such as `Asc`'s NPU manager), the C++ engine cannot use a standard single-context initialization loop:

1. **Context Handle Array (`Qnn_ContextHandle_t`):** The driver initializer must allocate and track an array of context handles:

```
Qnn_ContextHandle_t http_contexts[2]; // Index 0: HTP0 (PD 1), Index 1: HTP1 (PD 2)
```

2. **Discrete Shared Memory Allocations (`ion / memfd`):**
 - Shared memory buffers backing weight maps and activation spaces must be registered as discrete anonymous memory blocks (`ASharedMemory_create` or `memfd_create`) capped under 3.5GB per domain:

```
// Domain 0 allocation (Layers 0-24)
```

```
int fd_http0 = ASharedMemory_create("http0_tensor_pool", 3000 * 1024 * 1024);
```

```
// Domain 1 allocation (Layers 25-48)
```

```
int fd_http1 = ASharedMemory_create("http1_tensor_pool", 3000 * 1024 * 1024);
```

3. **FastRPC Binding:** FastRPC endpoints must explicitly map each file descriptor into its respective Hexagon Process Domain before layer execution begins.